

Edit by chaoweic

```

int dist[N];
void bfs(int s){
    memset(dist,-1,sizeof(dist));
    queue<int> q;
    q.push(s);
    dist[s]=0;
    while(q.size()){
        int u=q.front();
        q.pop();
        for(int v:g[u]){
            if(dist[v]==-1){
                dist[v]=dist[u]+1;
                q.push(v);
            }
        }
    }
}

//无权最短路
int n,m;
char g[N][N];
int dx[]={0,-1,0,1};
int dy[]={1,0,-1,0};
int step[N][N];

int bfs1(int sx,int sy,int ex,int ey){
    memset(step,-1,sizeof(step));
    queue<pii> q;
    q.push({sx,sy});
    step[sx][sy]=0;
    while(q.size()){
        auto [x,y]=q.front();q.pop();
        if(x==ex && y==ey) return step[x][y];
        req(i,0,3){
            int nx=x+dx[i],ny=y+dy[i];
            if(nx<0||nx>=n||ny<0||ny>=m) continue;
            if(g[nx][ny]!='#' && step[nx][ny]==-1){
                step[nx][ny]=step[x][y]+1;
                q.push({nx,ny});
            }
        }
    }
}

return -1;

```

```

}
//网格最短路
int bfs01(int sx,int sy){
    deque<pii> q;
    dist[sx][sy]=0;
    q.push_front({sx,sy});
    while(q.size()){
        auto [x,y]=q.front();q.pop_front();
        req(i,0,3){
            int nx=x+dx[i],ny=y+dy[i];
            if(nx<1 || nx>n || ny<1 || ny>m || g[nx][ny]=='*') continue;
            int w=(i==1)?1:0;
            if(dist[nx][ny]>dist[x][y]+w){//松弛
                dist[nx][ny]=dist[x][y]+w;
                if(w==0) q.push_front({nx,ny});
                else q.push_back({nx,ny});
            }
        }
    }
}

int d1[N][N],d2[N][N];
int bibfs(int sx,int sy,int ex,int ey){
    queue<pii> q1,q2;
    q1.push({sx,sy});
    q2.push({ex,ey});
    memset(d1,-1,sizeof(d1));
    memset(d2,-1,sizeof(d2));
    d1[sx][sy]=0;
    d2[ex][ey]=0;
    while(q1.size() && q2.size()){
        if(q1.size()<=q2.size()){
            auto [x,y]=q1.front();q1.pop();
            req(i,0,3){
                int nx=x+dx[i],ny=y+dy[i];
                if(nx<1 || nx>n || ny<1 || ny>m || g[nx][ny]=='*') continue;
                if(d1[nx][ny]==-1){
                    d1[nx][ny]=d1[x][y]+1;
                }
                if(d2[nx][ny]!=-1){
                    return d1[nx][ny]+d2[nx][ny];
                }
            }
        }
        else{

```

```

        auto [x,y]=q2.front();q2.pop();
        req(i,0,3){
            int nx=x+dx[i],ny=y+dy[i];
            if(nx<1 || nx>n || ny<1 || ny>m || g[nx][ny]=='*') continue;
            if(d2[nx][ny]==-1){
                d2[nx][ny]=d2[x][y]+1;
            }
            if(d1[nx][ny]!=-1){
                return d1[nx][ny]+d2[nx][ny];
            }
        }
    }
}
return -1;
}
//双向BFS
const long long INF=1e18;
void dij(int s,vector<int> pre){
    fill(dist,dist+N,INF);
    dist[s]=0;
    priority_queue<pii,vector<pii>,greater<>> pq;
    pq.push({0,s}); //距离, 点
    while(pq.size()){
        auto [d,u]=pq.top();pq.pop();
        if(d>dist[u]) continue; //剪枝
        for(auto [v,w]:g[u]){
            if(dist[u]+w<dist[v]){
                dist[v]=dist[u]+w;
                pre[v]=u;
                pq.push({dist[v],v});
            }
        }
    }
}

void road(int s,int e,vector<int>& pre){
    vector<int> ans;
    int cur=e;
    while(cur!=s){
        ans.pb(pre[cur]);
        cur=pre[cur];
    }
    ans.pb(s);
}

```

```

reverse(all(ans));
req(i,0,ans.size()-1) cout<<ans[i]<<" ";
}
//带路径回溯Dij
bool spfa(int s){
    vector<i64> dist(n+1,INF);
    vector<int> cnt(n+1,0); //入队次数
    vector<bool> inq(n+1,0); //判断要不要入队
    queue<int> q;

    dist[s]=0;
    q.push(s);
    inq[s]=1;

    while(q.size()){
        int u=q.front();q.pop();
        inq[u]=0;
        for(auto [v,w]:g[u]){
            if(dist[u]+w<dist[v]){
                dist[v]=dist[u]+w;
                cnt[v]++;
                if(cnt[v]>=n) return 0;
                if(!inq[v]){
                    q.push(v);
                    inq[v]=1;
                }
            }
        }
    }
    return 1;
}
//SPFA判负环
struct DSU{
    vector<int> fa,sz;
    DSU(int n){fa.resize(n+1);sz.assign(n+1,1);iota(all(fa),0);}
    int find(int x){return fa[x]==x?x:fa[x]=find(fa[x]);}
    bool unite(int a,int b){
        a=find(a),b=find(b);
        if(a==b) return 0;
        if(sz[a]<sz[b]) swap(a,b);
        fa[b]=a;sz[a]+=sz[b];
        return 1;
    }
}

```

```

};

struct Edge{
    int u,v,w;
    bool operator<(const Edge& other) const {return w<other.w;}
};

int kruskal(int n,vector<Edge>& edges){
    sort(all(edges));
    DSU dsu(n);
    int cost=0,cnt=0;
    for(auto &e:edges){
        if(dsu.unite(e.u,e.v)){
            cost+=e.w;
            if(++cnt==n-1) break;
        }
    }
    return (cnt==n-1)?cost:-1;
}

```

```

vector<int> fa(n+1);iota(all(fa),0);
int find(int x){return fa[x]==x?x:fa[x]=find(fa[x]);}
void unite(int a,int b){fa[find(a)]=find(b);}
//并查集ez版

```

```

int dfn[N],low[N],timer,cnt;
stack<int> st;
bool in[N];
void tarjan(int u){
    dfn[u]=low[u]=++timer;
    st.push(u);in[u]=1;
    for(int v:g[u]){
        if(!dfn[v]){
            tarjan(v);
            low[u]=min(low[u],low[v]);
        }else if(in[v]) low[u]=min(low[u],dfn[v]);
    }
    if(low[u]==dfn[u]){
        cnt++;
        while(1){
            int v=st.top();st.pop();
            in[v]=0;
            scc[v]=cnt;
        }
    }
}

```

```

        if(v==u) break;
    }
}
}
//tarjan求强连通分量

//Khan拓扑排序
bool Khan(vector<int>& topo){
    queue<int> q;
    req(i,1,n){
        if(in[i]==0) q.push(i); //入度为0入队
    }
    while(q.size()){
        int u=q.front();q.pop();
        topo.pb(u);
        for(int v:g[u]){
            if(--in[v]==0) q.push(v);
        }
    }
    if(topo.size()<n) return 1;
    else return 0;
}

void dfs(int u){
    vis[u]=1;
    for(int v:g[u]){
        if(!vis[v]) dfs(v);
    }
    //回溯
}

//排列枚举
int a[N],use[N],n;
void dfs(int depth){
    if(depth==n){
        req(i,0,n-1) cout<<a[i]<<((i==n-1)?'\n':" ");
        return;
    }
    req(i,1,n){
        if(!use[i]){
            use[i]=1;
            a[depth]=i;
            dfs(depth+1);
        }
    }
}

```

```

        use[i]=0;
    }
}

//树上遍历
void DfsTree(int u,int fa){
    for(int v:g[u]){
        if(v!=fa) dfsTree(v,u);
    }
}

//子集生成
int arr[N],n;
vector<int> cur;
void DfsSubset(int idx){
    if(idx==n){
        for(int x:cur) cout<<x<<((i==n-1)?'\n':" ");
        return;
    }
    DfsSubset(idx+1); //不选
    cur.pb(arr[idx]); //选
    DfsSubset(idx+1);
    cur.pop_back(); //回溯
}

struct ST{
    int n;vector<vector<int>> t;
    vector<int> lg;
    ST(vector<int>& a):n(a.size()),t(20,vector<int>(a.size())),lg(a.size()+1){
        req(i,2,n) lg[i]=lg[i>>1]+1;
        t[0]=a;
        for(int k=1;(1<<k)<=n;++k){
            for(int i=0;i+(i<<k)-1<n;++i)
                t[k][i]=min(t[k-1][i],t[k-1][i+(1<<(k-1))]);
        }
        int query(int l,int r){
            int k=lg[r-l+1];
            return min(t[k][l],t[k][r-(1<<k)+1]);
        }
    };
    ST st(a);
    int ans=st.query(1,5); //查询min

```


//ST表

```
struct TreeNode{
    int val;
    TreeNode *l,*r;
    TreeNode(int x):val(x),l(nullptr),r(nullptr){}
};
```

```
struct BIT{
    int n;
    vector<ll> t;
    BIT(int n):n(n),t(n+1,0){}
    void update(int i,ll v){
        for(;i<=n;i+=i&-i) t[i]+=v;
    }
    ll query(int i){
        ll s=0;
        for(;i>0;i-=i&-i) s+=t[i];
        return s;
    }
    ll query(int l,int r) return query(r)-query(l-1);
}
```

//树状数组

```
BIT bit(n);
bit.update(3,5)//位置3 +5
ll ans=bit.query(2,4)//查询2, 4区间和
```

```
struct SegTree{
    int n;
    struct Node{ll sum,lazy;};
    vector<Node> t;
    SegTree(int n):n(n),t(4*n+4){}
    void build(const vector<ll>& arr){
        build(1,1,n,arr);
    }
    //区间加
    void update(int L,int R,ll v,int o=1,int l=1,int r=-1){
        if(r==-1) r=n;
        if(L>r || R<l) return;
        if(L<=l && r<=R){apply(o,l,r,v);return;}
        pushdown(o,l,r);
        int mid=(l+r)>>1;
        update(L,R,v,o*2,l,mid);
    }
}
```

```

        update(L,R,v,o*2+1,mid+1,r);
        pushup(o);
    }
    //区间查询和
    ll query(int L,int R,int o=1,int l=1,int r=-1){
        if(r==-1) r=n;
        if(L>r || R<l) return 0;
        if(L<=l && r<=R) return t[o].sum;
        pushdown(o,l,r);
        int mid=(l+r)>>1;
        return query(L,R,o*2,l,mid)+query(L,R,o*2+1,mid+1,r);
    }
private:
    void pushup(int o) t[o].sum=t[o*2].sum+t[o*2+1].sum;
    void pushdown(int o,int l,int r){
        if(!t[o].lazy) return;
        int mid=(l+r)>>1;
        apply(o*2,l,mid,t[o].lazy);
        apply(o*2+1,mid+1,r,t[o].lazy);
        t[o].lazy=0;
    }
    void apply(int o,int l,int r,ll v){
        t[o].sum+=v*(r-l+1);
        t[o].lazy+=v;
    }
    void build(int o,int l,int r,const vector<ll>& arr){
        if(l==r){
            t[o].sum=arr[l];
            return;
        }
        int mid=(l+r)>>1;
        build(o*2,l,mid,arr);
        build(o*2+1,mid+1,r,arr);
        pushup(o);
    }
};
//线段树来了
SegTree seg(n);
seg.build(a);
seg.update(1,5,3);
ll ans=seg.query(2,4);

```

//多元素优先队列

```

auto type=tuple<int,int>;
priority_queue<type,vector<type>,Compare> pq;
struct Compare{
    bool operator()(const tuple<int,int>& a,const tuple<int,int>& b){
        if(get<0>(a)<get<0>(b)) return get<0>(a) < get<0>(b);
        return get<1>(a) > get<1>(b);
    }
};

```

```

int l=1,r=n,ans=n;

```

```

while(l<=r){
    int mid=(l+r)>>1;
    if(check(mid)){
        ans=mid;
        r=mid-1;
    }else l=mid+1;
}

```

//找最小值

```

int pos=lower_bound(all(a),x)-a.begin();//第一个>=x的元素的下标

```

```

vector<int> color(n,-1);

```

```

bool bio=1;

```

```

fuction<void(int,int)> bfs=[&](int s,int c){
    queue<int> q;
    q.push(s);color[s]=c;
    while(q.size()){
        int u=q.front();q.pop();
        for(int v:g[u]){
            if(color[v]==-1){
                color[v]=c^^1;
                q.push(v);
            }else if(color[v]==color[u])bio=0;
        }
    }
}

```

//二分图判定

```

using ULL=unsigned long long;

```

```

const ULL base=131;

```

```

const int N=1e6+5;

```

```

ULL h[N],p[N];

```

```

void init(const string& s){

```

```

    p[0]=1;

```

```

    req(i,1,s.size()){

```

```

        h[i]=h[i-1]*base+s[i-1];
        p[i]=p[i-1]*base;
    }
}
ULL getHash(int l,int r){
    return h[r]-h[l-1]*p[r-l+1];
}
getHash(l1,r1)==getHash(l2,r2);
int lcp(int i,int j){//二分求最长公共前缀
    int l=0,r=min(n-i+1,n-j+1),ans=0;
    while(l<=r){
        int mid=(l+r)>>1;
        if(getHash(i,i+mid-1)==getHash(j,j+mid-1)){
            ans=mid,l=mid+1;
        }else r=mid-1;
    }
    return ans;
}
//字符哈希

vector<int> Next;
void GetNext(const string& p){
    int len=p.size();
    Next.resize(len);
    int j=0,k=-1;
    Next[0]=-1;
    while(j<len-1){
        if(k==-1 || p[k]==p[j]){
            j++;k++;
            Next[j]=k;
        }else k=Next[k];
    }
}
int KMP(const string& t,const string& p){//在t中找p，返回首次出现位置
    int n=t.size(),m=p.size();
    if(m==0) return 0;
    GetNext(p);
    int i=0,j=0;
    while(i<n && j<m){
        if(j==-1 || t[i]==p[j]){
            i++;j++;
        }else j=Next[j];
    }
}

```

```

        return (j==m)?i-j:-1;
    }
vector<int> FindAllPos(const string& t,const string& p){//找所有出现位置
    vector<int> pos;
    int n=t.size(),m=p.size();
    GetNext(p);
    int i=0,j=0;
    while(i<n){
        if(j== -1 || t[i]==p[j]){
            i++;j++;
        }else j=Next[j];
        if(j==m){
            pos.pb(i-j);
            j=Next[j-1];
        }
    }
    return pos;
}
//KMP

```

```

vector<int> prefix(const string& s){
    int n=s.size();
    vector<int> pi(n);
    req(i,1,n-1){
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
        pi[i]=j;
    }
    return pi;
}
vector<int> FindAllPos(const string& t,const string& p){
    vector<int> pi=prefix(p),pos;
    int j=0;
    req(i,0,t.size()-1){
        while(j>0 && t[i]!=p[j]) j=pi[j-1];
        if(t[i]==p[j]) j++;
        if(j==p.size()){
            pos.pb(i-j+1);
            j=pi[j-1];
        }
    }
    return pos;
}

```

```
}  
//现代的前缀函数  $\pi$  写法
```

```
vector<int> manacher(const string& s){//返回每个位置的回文半径  
    string t="#";  
    for(char c:s){t+=c;t+='#';}  
    int n=t.size();  
    vector<int> p(n);  
    for(int i=0,c=0,r=0;i<n;++i){  
        p[i]=(i<r)?min(r-i,p[2*c-i]):1;  
        while(i-p[i]>=0 && i+p[i]<n && t[i-p[i]]==t[i+p[i]]) p[i]++;  
        if(i+p[i]>r){c=i;r=i+p[i];}  
    }  
    int mx=*max_element(all(p))-1;  
    return p;  
}  
//最长回文子串
```

```
const int MAXN=1e5+5;//总字符上限  
int trie[MAXN][26];  
int cnt[MAXN];  
int tot=0;  
void insert(const string& s){  
    int u=0;  
    for(auto c:s){  
        int ch=c-'a';  
        if(!trie[u][ch]) trie[u][ch]=++tot;  
        u=trie[u][ch];  
    }  
    cnt[u]++;  
}  
int query(const string& s){  
    int u=0;  
    for (char c:s) {  
        int ch=c-'a';  
        if(!trie[u][ch]) return 0;  
        u=trie[u][ch];  
    }  
    return cnt[u]; // 返回出现次数  
}  
//Trie树
```

```
struct AC{
```

```

int trie[MAXN][26], fail[MAXN], cnt[MAXN], tot;
AC():tot(0){
    memset(trie,0,sizeof(trie));
    memset(fail,0,sizeof(fail));
    memset(cnt,0,sizeof(cnt));
}
void insert(const string& s){
    int u=0;
    for(auto c:s){
        int ch=c-'a';
        if(!trie[u][ch]) trie[u][ch]=++tot;
        u=trie[u][ch];
    }
    cnt[u]++;
}
void build(){
    queue<int> q;
    req(i,0,25){
        if(trie[0][i]){
            fail[trie[0][i]]=0;
            q.push(trie[0][i]);
        }
    }
    while(q.size()){
        int u=q.front();q.pop();
        req(j,0,25){
            if(trie[u][j]){
                fail[trie[u][j]]=trie[fail[u]][j];
                q.push(trie[u][j]);
            }else trie[u][j]=trie[fail[u]][j];
        }
    }
}
int query(const string& t){
    int u=0,ans=0;
    for(auto c:t){
        u=trie[u][c-'a'];
        for(int v=u;v&&cnt[v]!=-1;v=fail[v]){
            ans+=cnt[v];
            cnt[v]=-1;
        }
    }
}

```

```

};
int deg[MAXN], sum[MAXN];
void TopoQuery(const string& t) {
    int u=0;
    for(char c:t) {u=trie[u][c-'a'];sum[u]++;}
    queue<int> q;
    req(i,1,tot) deg[fail[i]]++;
    req(i,1,tot) if(deg[i]==0) q.push(i);
    while (q.size()) {
        int u=q.front(); q.pop();
        int v=fail[u]; sum[v]+=sum[u];
        if(--deg[v]==0) q.push(v);
    }
    // sum[id[s]] 就是模式串 s 在 t 中的出现次数
}
//AC自动机

```

```

vector<int> primes,minp(n+1);
req(i,2,n){
    if(!minp[i]){
        minp[i]=i;
        primes.pb(i);
    }
    for(auto p:primes){
        if(p>minp[i] || 1LL*i*p>n) break;
        minp[i*p]=p;
    }
}
//线性筛
vector<pii> factorize(int x, const vector<int>& minp){
    vector<pii> fac;
    while (x>1) {
        int p=minp[x], cnt=0;
        while(x%p==0) {
            x/=p;
            ++cnt;
        }
        fac.pb({p, cnt});
    }
    return fac;
}
//质因数分解

```



```

ll qpow(ll a,ll b,ll mod){//模数为质数
    ll res=1;
    while(b){
        if(b&1) res=res*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return res;
}
ll inv(ll a){return qpow(a,MOD-2,MOD);}
//费马小定理求逆元

```

```

const int N=260;//最大矩阵边长
struct Mat{
    int n,a[N][N];
    Mat(){}
    Mat(int n):n(n){memset(a,0,sizeof(a));}
    static Mat identity(int n){
        Mat I(n);
        req(i,1,n) I.a[i][i]=1;
        return I;
    }//单位矩阵
    Mat operator*(const Mat& b) const{
        assert(n==b.n){
            Mat res(n);
            req(i,1,n){
                req(k,1,n){
                    if(a[i][k]==0) continue;
                    req(j,1,n){
                        res.a[i][j]=(res.a[i][j]+a[i][k]*b.a[k][j])%mod;
                    }
                }
            }
            return res;
        }
    }//矩阵乘法
    Mat pow(ll exp)const{
        Mat res=identity(n);
        Mat base=*this;
        while(exp){
            if(exp&1) res=res*base;
            base=base*base;
            exp>>=1;
        }
    }
}

```

```

    }
    return res;
}
};

```

//矩阵快速幂

```

const int B=63;
struct LinearBasis{
    ll p[B+1];
    LinearBasis(){memset(p,0,sizeof(p));}
    bool insert(ll x){
        for(int i=B;i>=0;--i){
            if(!(x>>i&1)) continue;
            if(!p[i]){p[i]=x;return 1;}
            x^=p[i];
        }
        return 0;
    }
    ll queryMax(){
        ll res=0;
        for(int i=B;i>=0;--i){
            if((res^p[i])>res) res^=p[i];
        }
        return res;
    }
    ll queryMin(){
        req(i,0,63){
            if(p[i]) return p[i];
        }
        return 0;
    }
};

```

//异或线性基

```

ll fac[N],invf[N];
void init(int n){
    fac[0]=1;
    req(i,1,n) fac[i]=fac[i-1]*i%mod;
    invf[n]=inv(fac[n]);
    for(int i=n-1;i>=0;--i)
        invf[i]=invf[i+1]*(i+1)%mod;
}
ll C(int n,int k){//C(n,k)

```

```

    if(k<0 || k>n) return 0;
    return fac[n]*invf[k]%mod*invf[n-k]%mod;
}
ll A(int n,int k){
    if(k<0 || k>n) return 0;
    return fac[n]*invf[n-k]%mod;
}

```

//排列组合数

```

int CountCoprime(int n,int m){
    vector<int> p;
    int tmp=m;
    for(int i=2;i*i<=tmp;++i){
        if(tmp%i==0){
            p.pb(i);
            while(tmp%i==0) tmp/=i;
        }
    }
    if(tmp>1) p.pb(tmp);
    int ans=0,sz=p.size();
    for(int mask=1;mask<(1<<sz);++mask){
        int bit=__builtin_popcount(mask),prod=1;
        req(i,0,sz-1)
        if(mask>>i&1) prod*=p[i];
        if(bit%2==1) ans+=n/prod;
        else ans-=n/prod;
    }
    return n-ans;
}

```

//容斥原理求[1,n]与m互斥的数

```

ll catalan(int n){return C(2*n,n)*inv(n+1)%mod;}
//卡特兰数

```

```

vector<vector<ll>> stirling(int n,int k){
    vector S(n+1,vector<ll>(k+1));
    S[0][0]=1;
    req(i,1,n){
        req(j,1,min(i,k)){
            S[i][j]=(S[i-1][j-1]+j*S[i-1][j])%mod;
        }
    }
    return S;
}

```

```

}
//斯特林数

using db=double;
const db EPS=1e-9;//误差
int sgn(db x){return (x>EPS)-(x<-EPS);}
struct Point{
    db x,y;
    Point(){}
    Point(db x,db y):x(x),y(y){}
    Point operator+(const Point& b) const{return{x+b.x,y+b.y};}
    Point operator-(const Point& b) const{return{x-b.x,y-b.y};}
    Point operator*(db k) const{return{x*k,y*k};}
    db cross(const Point& b) const{return x*b.y-y*b.x;}
    db dot(const Point& b) const{return x*b.x+y*b.y;}
    db len(){return hypot(x,y);}
};

db distToLine(const Point& p,const Point& a,const Point& b){
    return fabs(cross(b-a,p-a))/(b-a).len();
}
//点到直线距离
bool onSegment(const Point& p,const Point& a,const Point& b){
    return sgn(cross(a-p,b-p))==0 && sgn(dot(a-p,b-p))<=0;
}
//点是否在线段上

bool intersect(const Point& a1, const Point& a2,
               const Point& b1, const Point& b2) {
    auto d1=cross(b1-a1,a2-a1);
    auto d2=cross(b2-a1,a2-a1);
    auto d3=cross(a1-b1,b2-b1);
    auto d4=cross(a2-b1,b2-b1);
    // 严格相交（不含端点）
    if (sgn(d1)*sgn(d2)<0 && sgn(d3)*sgn(d4)<0) return true;
    // 端点共线情况
    if (sgn(d1)==0 && onSegment(b1,a1,a2)) return true;
    if (sgn(d2)==0 && onSegment(b2,a1,a2)) return true;
    if (sgn(d3)==0 && onSegment(a1,b1,b2)) return true;
    if (sgn(d4)==0 && onSegment(a2,b1,b2)) return true;
    return false;
}
//线段相交(含端点)

db polygonArea(const vector<Point>& p){
    db s=0;

```

```

    int n=p.size();
    req(i,0,n-1)
        s+=cross(p[i],p[(i+1)%n]);
    return fabs(s)/2;
} //多边形面积

```

```

fixed<<setprecision(10)//精度控制

```

```

(x >> k) & 1 // 获取 x 的第 k 位 (从0开始)
x |= (1 << k) // 将 x 的第 k 位设为 1
x &= ~(1 << k) // 将 x 的第 k 位设为 0
x ^= (1 << k) // 将 x 的第 k 位取反
x & -x // 获取 x 的最低位的 1 (lowbit)
x & (x - 1) // 将 x 的最低位的 1 置为 0
x > 0 && (x & (x - 1)) == 0 // 判断 x 是否为 2 的幂

```

```

// 获取 x 的二进制中 1 的个数 (popcount)
__builtin_popcount(x) // GCC内置, int
__builtin_popcountll(x) // long long
// 获取 x 的二进制末尾 0 的个数 (ctz)
__builtin_ctz(x)
// 获取 x 的二进制开头 0 的个数 (clz)
__builtin_clz(x)
// 枚举 x 的所有子集
for (int sub = x; sub; sub = (sub - 1) & x) {
    // 处理子集 sub
}

```

```

// 枚举大小为 k 的所有子集 (Gosper's Hack)

```

```

int sub = (1 << k) - 1;
while (sub < (1 << n)) {
    // 处理子集 sub
    int c = sub & -sub;
    int r = sub + c;
    sub = ((r ^ sub) >> 2) / c | r;
}

```

```

//获取x二进制表示的位数

```

```

static inline int bit_width(int x) {
#ifdef _MSC_VER
    unsigned long index;
    return _BitScanReverse64(&index, x) ? (int)(index + 1) : 0;
#else
    return x ? (int)(sizeof(int) * 8 - __builtin_clzll(x)) : 0;
#endif
}

```

```
}
```

```
LL exgcd(LL a, LL b, LL &x, LL &y){  
    if(!b){ x=1; y=0; return a; }  
    LL x1,y1,g=exgcd(b,a%b,x1,y1);  
    x=y1; y=x1-(a/b)*y1;  
    return g;  
}
```

//扩展欧几里得

```
int diff[N],d[N][N];  
void add(int l,int r,int v){diff[l]+=v;diff[r+1]-=v;}  
void add(int x1,int y1,int x2,int y2,v){  
    d[x1][y1]+=v  
    d[x1][y2+1]-=v  
    d[x2+1][y1]-=v  
    d[x2+1][y2+1]+=v  
}
```

```
int id=(i-1)*m+j,i=(id-1)/m+1,j=(id-1)%m+1;  
//差分
```

```
vector<int> slide(const vector<int>& a,int k){  
    deque<int> dq;vector<int> res;  
    req(i,0,a.size()-1)  
    {  
        while(!dq.empty() && a[dq.back()]<=a[i]) dq.pop_back();  
        dq.push_back(i);  
        if(dq.front() <= i-k) dq.pop_front();  
        if(i>=k-1) res.push_back(a[dq.front()]);  
    }  
    return res;  
}
```

//滑动窗口max/min

```
vector<pii> idx,res;  
static bool cmp(pii& a,pii& b){return a.first<b.first;}  
ll intervals(){  
    sort(all(idx),cmp);  
    res.pb(idx[0]);  
    req(i,1,idx.size()-1){  
        if(res.back().second>=idx[i].first){//根据题目考虑  
            res.back().second=max(res.back().second,idx[i].second)  
        }else res.pb(idx[i]);  
    }
```

```

    }
    return (ll)res.size();
}
//区间合并

```

$$dp[i] = \sum_{j \in [l, r]} dp[j]$$

```

vector<ll> pref(n+1);
dp[0]=1;pref[0]=1;
ll sum=0;
int l=0;
req(r,1,n){
    sum+=a[r];
    while(sum>limit) {
        sum-=a[l+1];
        l++;
    }
    //dp[r]=dp[l]+dp[l+1]+...+dp[r-1]
    dp[r]=(pref[r-1]-(l>0?pref[l-1]:0)+MOD)%MOD;
    pref[r]=(pref[r-1]+dp[r])%MOD;
}
//前缀和优化

```

$$dp[i] = \max_{j \in [i-k, i-1]} (dp[j] + cost(j, i))$$

```

int jump(vector<int>& nums) {
    int n=nums.size();
    vector<int> dp(n, N/2);
    dp[0]=0;
    deque<int> q;
    q.push_back(0);

    req(i,1,n-1){
        while (!q.empty() && q.front()+nums[q.front()]<i)
            q.pop_front(); // 移除不可达的

        if (!q.empty())
            dp[i]=dp[q.front()]+1;

        while (!q.empty() && dp[i]<=dp[q.back()])
            q.pop_back(); // 维护单调性
        q.pb(i);
    }
    return dp[n-1];
}
//单调队列优化

```



```

getline(cin,s)//读取下一行到输入
erase(pos,len)//删除从pos开始len个字符
replace(pos,n,str)//将从pos开始到n个字符换成str
tolower/toupper(s[i])
substr(pos,n)//截取字符串
find(str,pos)//在pos开始找str无则返回string::npos
accumulate(all(a),x)//求和
fill(all(a),x)//初始化序列
max/min_element(all(a))//找最值
reverse(all(a))//反转
to_string(int a)//数字转字符串
sort(all(a));a.erase(unique(all(a), a.end()));//去重
//fuck unordered_map...
//常用函数

```

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using u64 = unsigned long long;

#define req(i, a, n) for (int i = a; i <= n; ++i)
#define rep(i,a,n) for(int i = a; i >= n; --i)
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
#define pb push_back
#define pii pair<int, int>
//#define int long long

const int mod=998244353,MOD=1e9+7,N=1e5+1,INF=1e18;
int dx[]={0,1,0,-1},dy[]={1,0,-1,0};

void solve() {

}

signed main() {
    #ifndef ONLINE_JUDGE
        freopen("in.txt", "r", stdin);
    #endif
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;

```

```
    while (t--)  
        solve();  
    return 0;  
}
```

//开火车